BBRES - RNG

Bit Based Randomized Entropy System-Scheduler Based Random Number Generator

Technical Documentation & License Agreement

Project	BBRES-RNG - Bit-Based Randomized Entropy System
Author	Mehul Singh
Version	1.0 / March 2026
Language	Java (JDK 8+)
License	Custom Restrictive License v1.0
Repository	https://github.com/singhmehul7783/Bit-Based-Randomized-Entropy-System-Scheduler-Based-RNG.git
Validation	30/30 Statistical Tests Passed

Developed by Mehul Singh
https://github.com/singhmehul7783
Version 1.0 - March 2026

Table of Contents

1. Introduction & Overview

BBRES-RNG (Bit Based Randomized Entropy System - Scheduler-Based Random Number Generator) is a novel Java-based random number generator that fundamentally departs from conventional pseudorandom approaches. Rather than relying on deterministic mathematical formulas seeded with an initial value, BBRES-RNG harvests genuine entropy from the non-deterministic behaviour of the operating system's thread scheduler.

Every value produced by BBRES-RNG is shaped by the live, non-deterministic state of the host machine - including CPU load, interrupt timing, memory pressure, and kernel-level scheduling decisions - rendering each output practically unrepeatable under normal operating conditions.

1.1 Motivation

Standard pseudorandom number generators (PRNGs) such as Linear Congruential Generators (LCGs) and Mersenne Twister are deterministic: identical seeds produce identical sequences. While suitable for simulation, they are inherently predictable. BBRES-RNG was designed to address this limitation by converting real-time OS scheduling chaos into a usable entropy source - conceptually analogous to hardware true RNGs, but implemented entirely in software.

1.2 Key Characteristics

- Non-deterministic by design - no seed, no repeatable sequence
- Multi-threaded architecture - race conditions are the entropy source
- Entropy harvested from OS thread-scheduling microsecond variance
- Bitwise mixing via XOR and bit shifts to aggregate raw timing data
- Passes 30/30 statistical tests including the NIST SP 800-22 suite
- Performance matches Java SecureRandom & outperforms Java Math.random()

1.3 Comparison with Traditional PRNGs

Property	Traditional PRNG	BBRES-RNG
Entropy Source	Mathematical seed	OS thread scheduler
Determinism	Deterministic	Non-deterministic
Reproducibility	Reproducible (same seed)	Practically unrepeatable
Architecture	Single-threaded	Multi-threaded concurrency
Algorithm basis	LCG, Mersenne Twister, etc.	Timing noise + bitwise mixing

2. System Architecture

BBRES-RNG operates through a three-stage pipeline that transforms OS-level scheduling non-determinism into high-quality random bits. The architecture is entirely implemented in Java and requires no external hardware or OS-level entropy interfaces.

2.1 Pipeline Overview

Stage 1	**Thread Spawning & Race Conditions** n worker threads launched concurrently. OS scheduler determines execution order, creating an inherently unpredictable interleaving.
Stage 2	**Timing-Based Entropy Collection** Each thread records its scheduler-assigned execution position. Microsecond-level variance between thread timings becomes raw entropy.
Stage 3	**Bitwise Mixing & Output** XOR + bit-shift cascade aggregates timing results into a single random bit. Bits accumulate to form the final random integer.

2.2 Stage 1 - Thread Spawning & Controlled Race Conditions

A configurable number n of lightweight worker threads are launched simultaneously. The OS scheduler determines the order in which these threads proceed-a decision driven by real-time CPU state, system load, and kernel internals that are inherently unpredictable. The intentional concurrency creates controlled race conditions whose outcomes serve as the primary entropy source.

Default concurrency: `n = 71` threads. Valid range: `3 ≤ n ≤ 1000`.

2.3 Stage 2 - Timing-Based Entropy Collection

Each worker thread captures its execution-order position with microsecond-level precision using a synchronized flag array. The sequence in which threads write to this array-determined purely by scheduler timing-constitutes the raw entropy input. The top 20% and bottom 20% of the ordering are selected to maximise entropy variance.

2.4 Stage 3 - Bitwise Mixing & Aggregation

Raw timing results are aggregated across thread groups using XOR operations and bit-shift sequences to produce a single random bit per cycle. The mixing function eliminates structural bias and ensures uniform bit distribution.

XOR cascade: `xor ^= xor << 10; xor ^= xor >>> 23; xor ^= xor << 7;`

Bits are assembled one at a time until the required binary length for the target range is reached. The resulting binary string is converted to a long integer and validated against the requested range via rejection sampling.

2.5 Generation Techniques

Technique G1 - Direct Sequential

Threads are assigned to groups of 32 and launched directly. No pre-shuffling is performed. Suitable for general-purpose use with predictable thread-group allocation.

Technique G2 - Shuffled Assignment

Thread IDs are pre-shuffled using a Fisher-Yates algorithm (backed internally by BBRES v1 for the shuffle step) before group assignment. This introduces an additional layer of non-determinism into the thread-to-group mapping, further strengthening entropy at the cost of slightly higher computation.

3. API Reference

All public functionality is exposed through the RNG class in the bbresRNG package. The class is instantiated once and its methods called as needed.

3.1 Class: RNG

Package: `bbresRNG` · **Import:** `import bbresRNG.RNG;`

3.2 Method Signatures

Method Signature	Description
`bbresRNG()`	Returns random long in [0, 10] with n=71, Technique G1.
`bbresRNG(max)`	Returns random long in [0, max] with defaults.
`bbresRNG(min, max)`	Returns random long in [min, max] with n=71, G1.
`bbresRNG(min, max, n)`	Returns random long in [min, max] with n threads, G1.
`bbresRNG(min, max, n, randTech)`	Full control over range, concurrency, and technique.

3.3 Parameters

Parameter	Type	Default	Description
`min`	long	`0`	Lower bound of the output range (inclusive).
`max`	long	`10`	Upper bound of the output range (inclusive).
`n`	int	`71`	Number of concurrent worker threads. Must satisfy 3 ≤ n ≤ 1000. Higher values increase entropy diversity at the cost of execution time.
`randTech`	int	`1`	Bit generation technique. 1 = G1 (direct), 2 = G2 (shuffled). G2 provides additional pre-shuffle non-determinism.

3.4 Return Value

All overloads return a long integer in the closed interval [min, max]. The implementation uses rejection sampling to ensure strict range compliance with zero bias.

3.5 Input Validation & Defaults

- If n ≤ 2 or n > 1000, n is silently reset to the default value of 71 and a warning is printed to stdout.
- If randTech is not 1 or 2, randTech is reset to 1 and a warning is printed.
- If min > max, an IllegalArgumentException (RuntimeException) is thrown with message "Invalid range".
- If min == max, the method returns min immediately without any thread activity.

3.6 Usage Examples

```
import bbresRNG.RNG;

public class Example {
    public static void main(String[] args) {
        RNG rng = new RNG();

        // Default: random integer in [0, 10]
        long v1 = rng.bbresRNG();

        // Custom range: [5, 15]
        long v2 = rng.bbresRNG(5, 15);

        // Custom concurrency: n=50 threads, range [1, 100]
        long v3 = rng.bbresRNG(1, 100, 50);

        // Full control: range [0, 10000], n=35, Technique G2
        long v4 = rng.bbresRNG(0, 10000, 35, 2);
    }
}
```

4. Installation & Usage

4.1 Prerequisites

- Java Development Kit (JDK) 8 or later
- Any terminal emulator or Java IDE (IntelliJ IDEA, Eclipse, VS Code, etc.)
- Git (optional, for cloning)

4.2 Obtaining the Source

```
# Clone the repository
git clone https://github.com/singhmehul7783/Bit-Based-Randomized-Entropy-System-Scheduler-Based-RNG.git


# Navigate to the project root
cd Bit-Based-Randomized-Entropy-System-Scheduler-Based-RNG
```

Alternatively, download the ZIP archive directly from the GitHub repository and extract it to a directory of your choice.

4.3 Compilation

```
# Compile from the project root (src/ as source root)
javac -sourcepath src -d out src/Main.java


# Or compile individual packages
javac src/bbresRNG/*.java src/bbresv1/*.java src/Main.java
```

4.4 Execution

```
# Run the bundled demonstration
java -cp out Main


# Expected output (values vary each run)
Random number between 0 and 10: 7
Random number between 5 and 15: 12
Random number between 1 and 100 with n=50: 63
...
```

4.5 Project Structure

Path	Description
`src/Main.java`	Entry point - bundled usage demonstration
`src/bbresRNG/RNG.java`	Public API - all user-facing method overloads
`src/bbresRNG/randomBitGeneratorModifiedRoot.java`	Core orchestrator - G1/G2 bit generation logic
`src/bbresRNG/modRandomBitGenRNG.java`	Segment thread - manages a group of worker threads
`src/bbresRNG/randomBitGenRNG.java`	Worker thread - entropy harvesting via flag array
`src/bbresv1/RNGv1.java`	BBRES v1 - used internally by G2 shuffle step
`docs/`	Validation reports (PDF) and benchmark CSV data
`LICENSE`	Custom Restrictive License v1.0

5. Statistical Validation

BBRES-RNG was subjected to a rigorous 30-test validation battery evaluated against 2,000,000 bits and 100,000 integers sampled from the generator output. Results were benchmarked head-to-head against Java's two standard RNG implementations. All tests were conducted at significance level α = 0.01.

5.1 Scorecard

RNG System	Tests Passed	Verdict	Failed Tests
BBRES-RNG	**30 / 30**	**ACCEPTED**	None
Java SecureRandom	30 / 30	ACCEPTED	None
Java Math.random()	28 / 30	REVIEW NEEDED	Maurer's, Gap

5.2 NIST SP 800-22 Core Tests (9/9)

Test	p-value	Result
Frequency (Monobit)	0.094324	**PASS**
Block Frequency (M=128)	0.923799	**PASS**
Runs Test	0.323306	**PASS**
Longest Run of Ones	0.469160	**PASS**
Cumulative Sums (Forward)	0.178039	**PASS**
Cumulative Sums (Reverse)	0.163651	**PASS**
Approximate Entropy (m=2)	0.053664	**PASS**
Serial (m=2) - d1	0.151995	**PASS**
Serial (m=2) - d2	0.324972	**PASS**

5.3 Extended Uniformity Tests (6/6)

Test	p-value	Result
Maurer's Universal Statistical	0.981707	**PASS**
Poker Test (m=4)	0.479188	**PASS**
Random Excursion Variant	0.500000	**PASS**
Byte-Level Chi-Square	0.468109	**PASS**

Nibble-Level Chi-Square	0.479188	**PASS**
2-Bit Pair Distribution	0.386897	**PASS**

5.4 Spectral, Adversarial & Cryptographic Tests (5/5)

Test	p-value	Result
DFT Periodicity	0.227460	**PASS**
Frequency Prediction (w=8)	0.884165	**PASS**
ML Attack - Logistic Regression (w=16)	0.579260	**PASS**
Pattern Repetition (w=53)	1.000000	**PASS**
SHA-256 CSPRNG Wrapper	1.000000	**PASS**

5.5 Integer Distribution & Sequence Tests (10/10)

Test	p-value	Result
Chi-Square Uniformity	0.475159	**PASS**
Mean (Z-test)	0.398873	**PASS**
Variance (Chi-Square)	0.842551	**PASS**
Binned Goodness-of-Fit	0.756309	**PASS**
Birthday Spacing	1.000000	**PASS**
Coupon Collector	0.500000	**PASS**
Runs Up/Down	0.994013	**PASS**
Lag-1 Autocorrelation	0.418097	**PASS**
Gap Test (KS)	0.332555	**PASS**
Permutation (t=5)	0.703417	**PASS**

5.6 Where Math.random() Failed

BBRES-RNG passed two tests that Java Math.random() failed, demonstrating its superior randomness quality over Java's default PRNG.

Test	Math.random()	BBRES-RNG	Why It Matters
Maurer's Universal Statistical	**FAIL p=0.0083**	**PASS p=0.9817**	Detects compressibility; reveals subtle LCG patterns in output.

Gap Test (KS)	**FAIL p=0.0005**	**PASS p=0.3326**	Non-uniform gap distribution between repeated values.

6. Entropy Quality Metrics

Entropy quality was measured across 2,000,000 bits of BBRES-RNG output and compared against both Java reference implementations. The following metrics characterise the information-theoretic quality of the generator.

6.1 Entropy Comparison

Metric	BBRES-RNG	SecureRandom	Math.random()	Theoretical Max
Shannon Entropy (8-bit)	**7.999260**	7.999311	7.999233	8.000000
Min-Entropy (8-bit)	**7.883082**	7.828281	7.879001	8.000000
Transition Rate	**0.500348**	0.499482	0.500284	0.500000
Bit Balance (1s: 0s)	**50.06: 49.94**	49.97: 50.03	50.02: 49.98	50: 50

6.2 Key Findings

- **Shannon Entropy:** 7.999260 bits/symbol - 99.991% of the theoretical maximum of 8.000000. Indicates near-perfect symbol distribution.
- **Min-Entropy:** 7.883082 - the highest among all three RNGs tested. Min-entropy quantifies worst-case unpredictability; BBRES-RNG's superior value means even the most predictable outputs remain highly uncertain.
- **Transition Rate:** 0.500348 - within 0.035% of the ideal 0.5. Bit-to-bit transitions are essentially coin-flip random.
- **Bit Balance:** 50.06% ones vs 49.94% zeros - indistinguishable from a fair coin at scale.

6.3 Autocorrelation

Lag	BBRES-RNG	SecureRandom	Math.random()
1	-0.0007	+0.0010	-0.0006
2	-0.0009	-0.0005	-0.0002
8	-0.0000	-0.0011	-0.0002
32	-0.0010	+0.0002	+0.0001
256	-0.0003	-0.0002	+0.0007

Autocorrelation values across all tested lags are within statistical noise (±0.001), confirming the absence of serial dependence in BBRES-RNG output streams.

7. Design Philosophy

Concurrency is inherently non-deterministic. The OS thread scheduler makes decisions based on CPU load, interrupt timing, memory pressure, and hundreds of other real-time factors that are impossible to predict or reproduce from user space. BBRES-RNG exploits this property deliberately and systematically.

By creating controlled race conditions and measuring their outcomes with microsecond precision, the system converts scheduling-level chaos into usable cryptographic-quality randomness. This approach is conceptually analogous to hardware true random number generators (TRNGs) that harvest physical noise from thermal, radioactive, or photonic phenomena-except that in BBRES-RNG, the noise source is the operating system itself.

7.1 The Entropy Source

The fundamental insight driving BBRES-RNG is that OS thread scheduling decisions are determined by a chaotic function of CPU state, memory hierarchy behaviour, kernel interrupt handling, and hardware interrupt timings. These factors collectively create microsecond-level variance in thread execution ordering that:

- Cannot be predicted in advance from user space
- Cannot be reproduced across runs even on the same hardware
- Varies with system load, making the output machine- and moment-specific
- Passes adversarial machine-learning tests with high confidence

7.2 Validation Methodology

The 30-test battery covers four distinct domains of randomness analysis: NIST SP 800-22 core tests (frequency, runs, serial correlation), extended uniformity tests (Maurer's compressibility, Poker, Chi-Square), spectral and adversarial tests (DFT periodicity, logistic regression ML attacks), and integer distribution tests (birthday spacing, coupon collection, permutation). Passing all 30 at $\alpha = 0.01$ provides strong statistical evidence that BBRES-RNG output is indistinguishable from ideal random sequences.

7.3 Scope & Limitations

BBRES-RNG is validated as a high-quality statistical RNG matching the quality of Java SecureRandom. Users should note the following:

- The validation reports are for informational purposes; they do not constitute formal NIST certification.
- Performance scales with n; higher concurrency values increase entropy diversity but also increase wall-clock execution time.
- The generator is not formally analysed for cryptographic hardness proofs. For security-critical applications requiring certified CSPRNGs, consult a security specialist.
- Output quality may vary under extreme system load where scheduler behaviour becomes more deterministic.

8. Class Reference

8.1 bbresRNG.RNG

The public entry point. Instantiate once per session; all state is reset on each call.

Field	Type	Description
`min`	long (static)	Lower bound for current generation call.
`max`	long (static)	Upper bound for current generation call.
`n`	int (static)	Worker thread count. Default: 71.
`randTech`	long (static)	Active technique selector (1 or 2).

8.2 bbresRNG.randomBitGeneratorModifiedRoot

Core orchestrator for single-bit generation. Contains the two generation technique implementations.

Method	Returns	Description
`generateRandBitG1()`	`String`	Direct thread group allocation. Returns single random bit as "0" or "1".
`generateRandBitG2()`	`String`	Shuffled assignment variant. Pre-shuffles thread IDs before group allocation.

8.3 bbresRNG.modRandomBitGenRNG

Segment thread that manages a sub-group of worker threads. Extends Thread. Collects and XOR-mixes the timing entropy from its assigned worker batch.

Field	Type	Description
`bitSelected[]`	int[64] (static volatile)	Array of XOR-aggregated bits from each segment thread.
`ids[]`	int[] (volatile)	Shuffled or direct ID array for worker assignment.
`localStart/localEnd`	int (volatile)	Index bounds of this segment's worker thread slice.

8.4 bbresRNG.randomBitGenRNG

Leaf worker thread. Extends Thread. Races to write its ID into the shared flag array; the order in which threads succeed forms the raw entropy.

Field	Type	Description
`flag []`	int [] (static volatile)	Shared ordering array - threads race to fill it. Size = n.
`conc`	AtomicIntegerArray (static volatile)	Synchronisation signal array; worker spins until parent segment sets its slot to 1.

9. License Agreement

BBRES-RNG Custom Restrictive License, Version 1.0 - March 2026

https://github.com/singhmehul7783/Bit-Based-Randomized-Entropy-System-Scheduler-Based-RNG

9.1 Definitions

For the purposes of this License, the following terms are defined as set out below.

- **"Software"** means the BBRES-RNG source code, object code, documentation, validation reports, test data, and all associated files contained in this repository.
- **"Author"** means Mehul Singh, the original creator and copyright holder.
- **"You" / "Your"** means any individual or legal entity exercising permissions granted under this License.
- **"Commercial Use"** means any use intended for or directed toward commercial advantage or monetary compensation, including incorporation into commercial products, paid services, or revenue-generating activities.
- **"Derivative Work"** means any work based on or derived from the Software, including modifications, translations, adaptations, extensions, ports, or any work incorporating substantial portions of the source code, algorithms, or architecture.
- **"Distribution"** means making the Software available to any third party by any means.

9.2 Grant of Limited Rights

Subject to the terms and conditions of this License, the Author grants You a non-exclusive, non-transferable, non-sublicensable, revocable, limited right to:

- VIEW and READ the source code for personal educational purposes and academic research.
- EXECUTE the Software in its unmodified form for personal study, academic research, or educational coursework.
- CITE and REFERENCE the Software, its architecture, validation results, and documentation in academic publications, research papers, theses, and educational materials, provided that proper attribution is given in accordance with Section 9.3.
- FORK the repository on GitHub solely for the purpose of submitting pull requests back to the original repository, subject to the Author's approval.

9.3 Attribution Requirements

Any permitted use under Section 9.2 must include the following attribution in a prominent location:

> **For academic papers and publications:** *"BBRES-RNG: Bit-Based Randomized Entropy System Scheduler-Based RNG, developed by Mehul Singh (2026). Source: https://github.com/singhmehul7783/Bit-Based-Randomized-Entropy-System-Scheduler-Based-RNG"*

For presentations, coursework, and other educational materials: credit must be given to Mehul Singh as the original author with a link to the original repository.

Removal, alteration, or obscuring of any copyright notices, license notices, or attribution statements is strictly prohibited.

9.4 Restrictions - Prohibited Actions

Unless You have obtained prior written permission from the Author, You MAY NOT:

- USE the Software for any Commercial Use.
- MODIFY, alter, adapt, translate, or create any Derivative Work based on the Software, in whole or in part.
- DISTRIBUTE, redistribute, publish, sublicense, sell, lease, rent, loan, or otherwise transfer the Software or any portion thereof to any third party.
- SUBLICENSE any rights granted under this License.
- INCORPORATE the Software or any substantial portion of its source code, algorithms, or architectural design into any other software project, product, library, or system.
- REVERSE ENGINEER, decompile, or disassemble any compiled or obfuscated portions for purposes other than those expressly permitted.
- REMOVE OR ALTER any copyright notices, license headers, author attributions, or proprietary markings.
- USE the Software to train, fine-tune, or otherwise develop machine learning models, artificial intelligence systems, or automated code generation tools without explicit written permission.
- CLAIM authorship or ownership of the Software, its algorithms, or its architectural design.
- FILE any patent application that covers or claims any aspect of the Software's algorithms, methods, architecture, or implementation.

9.5 Requesting Permission

To request permission for any use not expressly granted under Section 9.2, contact the Author with:

- Your full name and affiliation (individual or organization)
- A detailed description of the intended use
- The scope and duration of the requested permission
- Whether the use is commercial or non-commercial

Contact: https://github.com/singhmehul7783 · Email: as listed on the Author's GitHub profile

The Author reserves the sole and absolute right to grant or deny any permission request at their discretion. Silence or non-response shall NOT be construed as implicit permission.

9.6 Intellectual Property

The Software, including all source code, algorithms, architectural design, documentation, and validation methodology, is the exclusive intellectual property of the Author. This License does not transfer any ownership rights, title, or interest in the Software. No trademark, service mark, or trade name rights are granted under this License.

9.7 No Warranty

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NONINFRINGEMENT. THE AUTHOR MAKES NO WARRANTY THAT THE SOFTWARE IS ERROR-FREE, THAT IT WILL OPERATE WITHOUT INTERRUPTION, OR THAT IT IS SUITABLE FOR ANY PARTICULAR PURPOSE, INCLUDING CRYPTOGRAPHIC SECURITY. VALIDATION RESULTS ARE PROVIDED FOR INFORMATIONAL PURPOSES ONLY AND DO NOT CONSTITUTE A GUARANTEE OF CRYPTOGRAPHIC SECURITY OR COMPLIANCE WITH ANY STANDARD.

9.8 Limitation of Liability

IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY CLAIM, DAMAGES, OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT, OR OTHERWISE, ARISING FROM, OUT OF, OR IN CONNECTION WITH THE SOFTWARE OR ITS USE. THIS LIMITATION APPLIES TO ALL DAMAGES OF ANY KIND, INCLUDING DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES, LOSS OF PROFITS, LOSS OF DATA, OR BUSINESS INTERRUPTION.

9.9 Termination

- This License and the rights granted hereunder terminate automatically upon breach of any term.
- Upon termination, you must immediately cease all use and destroy all copies in Your possession.
- The Author reserves the right to revoke this License at any time, for any reason, with or without notice.
- Sections 9.6, 9.7, 9.8, and 9.10 survive termination.

9.10 Governing Law & Jurisdiction

This License shall be governed by and construed in accordance with the laws of India, without regard to conflict-of-law provisions. Any disputes shall be subject to the exclusive jurisdiction of the courts located in India.

9.11 Severability

If any provision of this License is held unenforceable or invalid, it shall be modified to the minimum extent necessary to make it enforceable. All remaining provisions shall continue in full force and effect.

9.12 Entire Agreement

This License constitutes the entire agreement between You and the Author with respect to the Software and supersedes all prior or contemporaneous understandings. No amendment shall be effective unless made in writing and signed by the Author.

Permission Requests

Complete the BBRES-RNG_Permission_Template.docx form (included in docs/form/) and send it to the Author via the public email listed on the GitHub profile below.

Mehul Singh - https://github.com/singhmehul7783